

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

« _____ »

Кафедра _____

ЛАБОРАТОРНАЯ РАБОТА № 1

по дисциплине «Основы алгоритмизации и программирование»

Итерационные методы вычисления элементарных функций:
квадратный корень, кубический корень и натуральный логарифм

Вариант 6

Направление: 38.03.05 «Бизнес-информатика»

Курс: 2

Выполнил: студент группы БИ 2-1

ФИО: Голдаева Валерия Валентиновна

Проверил: _____

2026

1. Цель работы и формулировка задания

Цель работы: сформировать представление об итерационных алгоритмах как о классе вычислительных процедур, результат которых достигается последовательным уточнением приближений, и освоить приёмы оценки их сходимости и трудоёмкости.

Задание варианта 6:

Вычислить $\sqrt{13}$, $\sqrt[3]{-27}$ и $\ln 7$ с точностью $\varepsilon = 10^{-5}$.

Параметры варианта:

- Критерий остановки для корней — Р (по разности соседних приближений $|x_{n+1} - x_n| < \varepsilon$);
- Начальное приближение: $x_0 = a/2$ (для кубического корня — $|b|/2$);
- Дополнительное требование Д1: вывести таблицу итераций для каждой функции и проанализировать изменение числа верных знаков.

Важно: внутри вычислительных функций запрещено использовать `math.sqrt`, `math.log`, `**0.5` и аналоги.

2. Краткие теоретические сведения

2.1. Понятие итерационного алгоритма

Итерационным называется алгоритм, в котором искомая величина x^* получается как предел последовательности приближений $x_0, x_1, x_2, \dots$, где $x_{n+1} = \varphi(x_n)$. Алгоритм определяется начальным приближением, итерационной формулой и критерием остановки.

2.2. Метод Ньютона

Для решения уравнения $F(x) = 0$ метод Ньютона строит последовательность:

$$x_{n+1} = x_n - F(x_n) / F'(x_n)$$

Метод обладает квадратичной сходимостью: число верных знаков примерно удваивается на каждой итерации.

Квадратный корень. $F(x) = x^2 - a \rightarrow$

$$x_{n+1} = \frac{1}{2} (x_n + a / x_n)$$

Кубический корень. $F(x) = x^3 - b \rightarrow$

$$x_{n+1} = \frac{1}{3} (2x_n + b / x_n^2)$$

Для отрицательного b : $\sqrt[3]{b} = -\sqrt[3]{|b|}$.

2.3. Натуральный логарифм через ряд

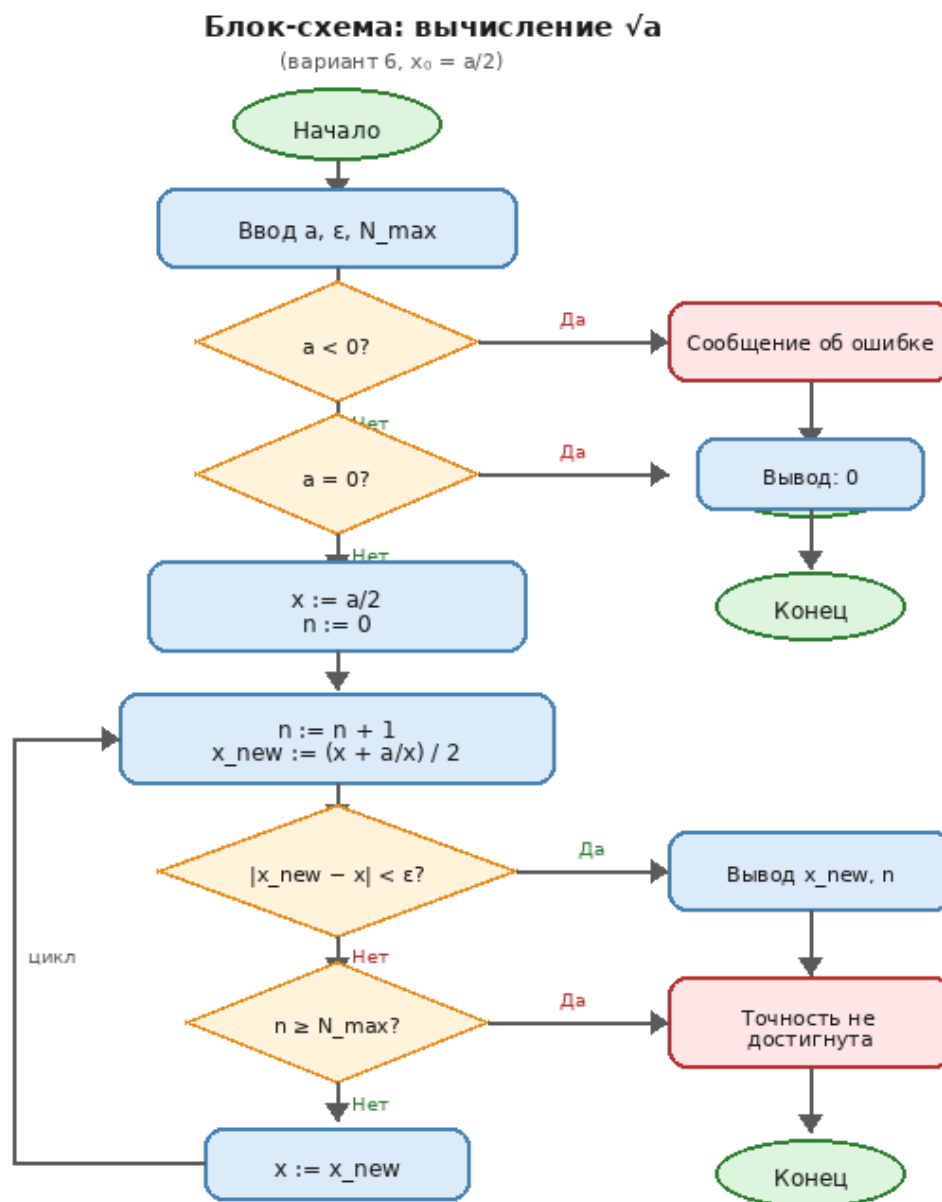
$$\ln c = 2 (y + y^3/3 + y^5/5 + \dots), y = (c - 1)/(c + 1)$$

Для любого $c > 0$ выполняется $|y| < 1$. Суммирование прекращается при $|t_k| < \varepsilon$. Сходимость линейная.

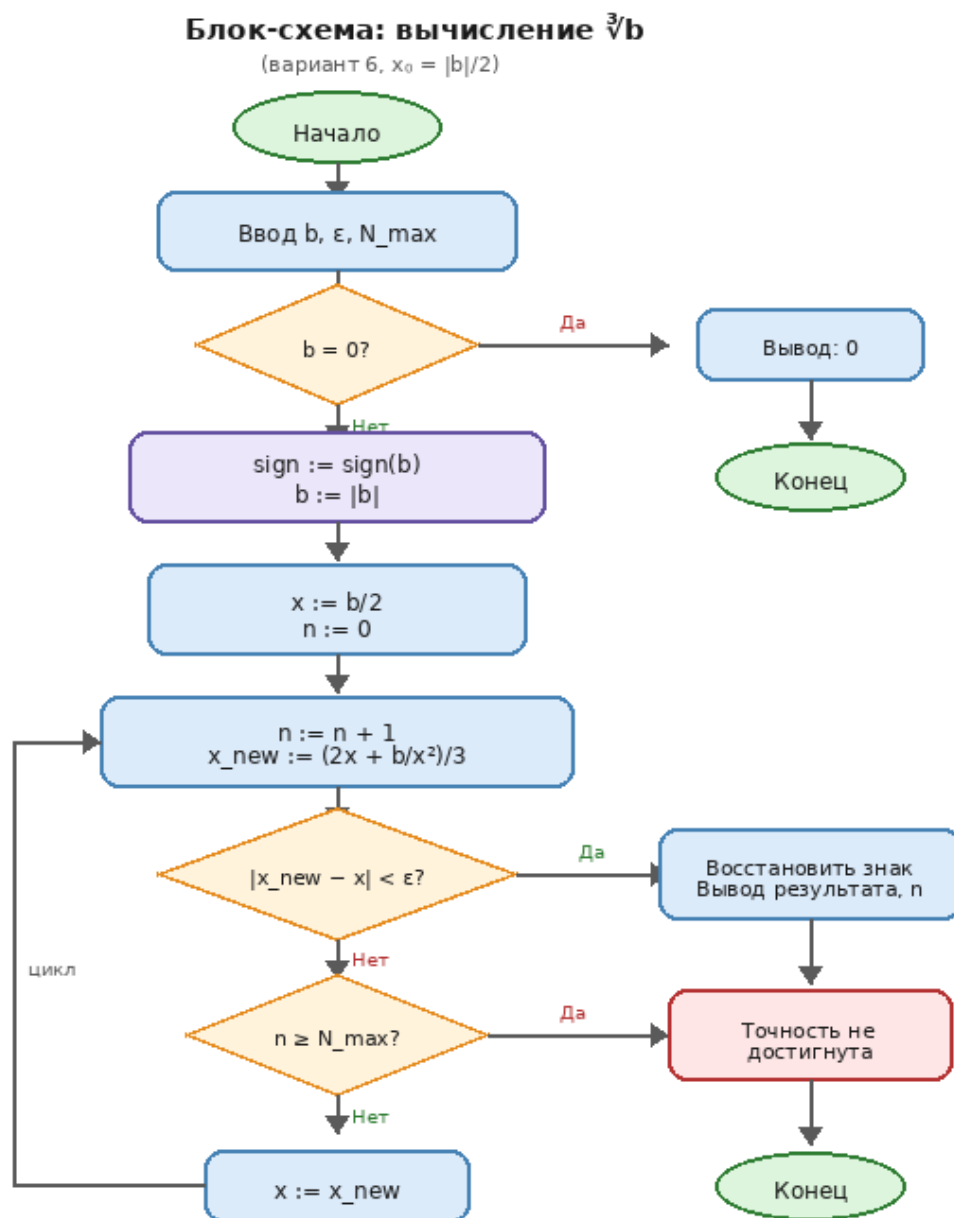
3. Блок-схемы алгоритмов

Ниже приведены блок-схемы алгоритмов в нотации, близкой к ГОСТ 19.701-90 (овалы — начало/конец, прямоугольники — процессы, ромбы — условия).

3.1. Блок-схема вычисления квадратного корня

Рисунок 1 — Блок-схема алгоритма вычисления $\sqrt{a}$ ($x_0 = a/2$)

3.2. Блок-схема вычисления кубического корня

Рисунок 2 — Блок-схема алгоритма вычисления $\sqrt[3]{b}$ ($x_0 = |b|/2$)

3.3. Блок-схема вычисления натурального логарифма

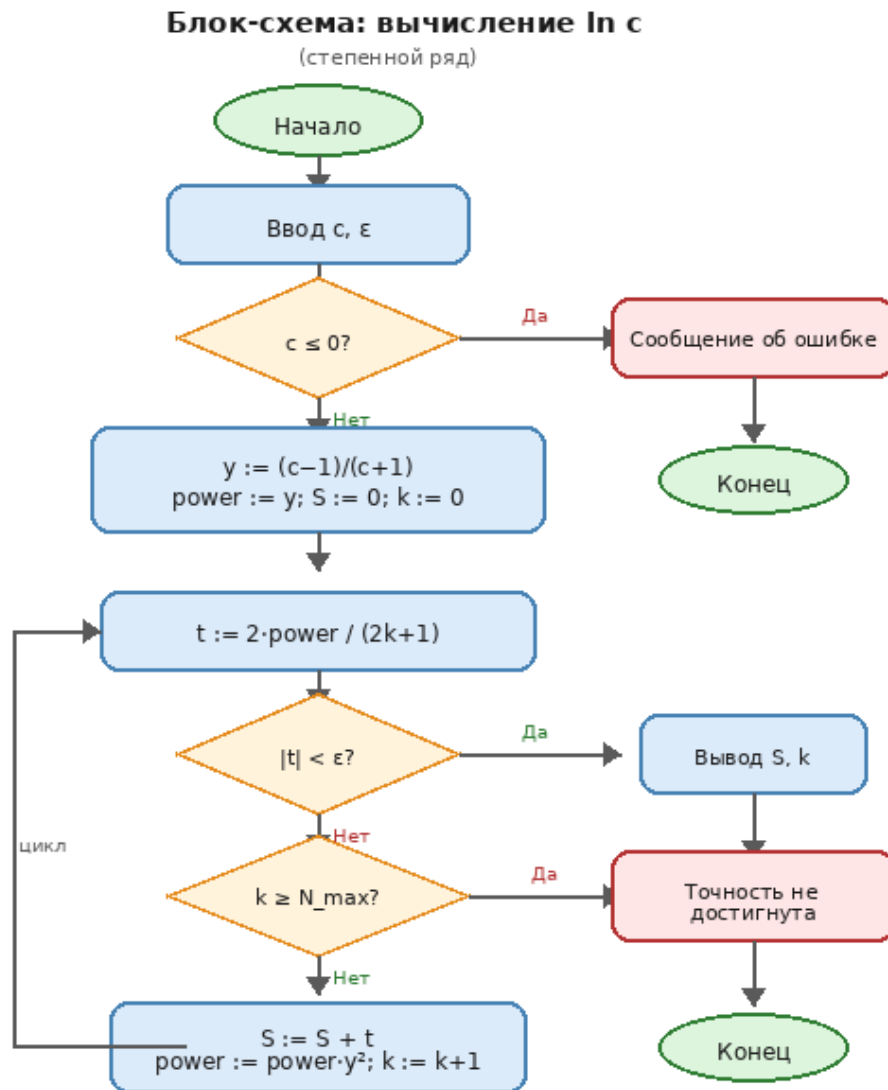


Рисунок 3 — Блок-схема алгоритма вычисления $\ln c$ (степенной ряд)

4. Листинг программы (ключевые функции)

Полный код — в файле main.py. В вычислительных функциях не используются `math.sqrt` / `math.log` / `**0.5`.

```

def sqrt_newton(a, eps=EPS, n_max=N_MAX):
    if a < 0: raise ValueError("a >= 0")
    if a == 0: return 0.0, 0
    x = a / 2.0          # x0 = a/2 (вариант 6)
    for n in range(1, n_max + 1):
        x_new = 0.5 * (x + a / x)
        if abs(x_new - x) < eps:
            return x_new, n
    x = x_new
  
```

```

raise RuntimeError("Точность не достигнута")

def cbrt_newton(b, eps=EPS, n_max=N_MAX):
    if b == 0: return 0.0, 0
    sign = -1 if b < 0 else 1
    b_abs = abs(b)
    x = b_abs / 2.0      # x0 = |b|/2
    for n in range(1, n_max + 1):
        x_new = (2 * x + b_abs / (x * x)) / 3
        if abs(x_new - x) < eps:
            return sign * x_new, n
        x = x_new
    raise RuntimeError("Точность не достигнута")

def ln_series(c, eps=EPS, n_max=10000):
    if c <= 0: raise ValueError("c > 0")
    y = (c - 1) / (c + 1)
    y2 = y * y; power = y; total = 0.0
    for k in range(n_max):
        term = 2 * power / (2 * k + 1)
        if abs(term) < eps:
            return total, k
        total += term
        power *= y2
    raise RuntimeError("Точность не достигнута")

```

5. Результаты расчётов

5.1. Итоговая таблица результатов ($\epsilon = 10^{-5}$)

Функция	Аргумент	Результат	Эталон	Погрешность	Итер.
$\sqrt{a}$	13	3.605551275464	3.605551275464	$4.44 \cdot 10^{-16}$	5
$\sqrt[3]{b}$	-27	-3.000000000000	-3.000000000000	$2.22 \cdot 10^{-15}$	8
$\ln c$	7	1.945891748543	1.945910149055	$1.84 \cdot 10^{-5}$	15

5.2. Таблица итераций для $\sqrt{13}$ (требование Д1)

$x_0 = 6.5$

n	x_n	$ x_{n+1} - x_n $	x_{n+1}
1	6.500000000000	$2.25 \cdot 10^0$	4.250000000000
2	4.250000000000	$5.96 \cdot 10^{-1}$	3.654411764706
3	3.654411764706	$4.85 \cdot 10^{-2}$	3.605877914546
4	3.605877914546	$3.27 \cdot 10^{-4}$	3.605551290258
5	3.605551290258	$1.48 \cdot 10^{-8}$	3.605551275464

5.3. Таблица итераций для $\sqrt[3]{(-27)}$

Работа с модулем, $x_0 = 13.5$; знак восстанавливается в конце

n	x_n	$ x_{n+1} - x_n $	x_{n+1}
1	13.500000000000	$4.45 \cdot 10^0$	9.049382716049
2	9.049382716049	$2.91 \cdot 10^0$	6.142823558176
3	6.142823558176	$1.81 \cdot 10^0$	4.333725614686
4	4.333725614686	$9.65 \cdot 10^{-1}$	3.368353585552
5	3.368353585552	$3.30 \cdot 10^{-1}$	3.038813723595
6	3.038813723595	$3.83 \cdot 10^{-2}$	3.000493643656
7	3.000493643656	$4.94 \cdot 10^{-4}$	3.000000081210
8	3.000000081210	$8.12 \cdot 10^{-8}$	3.000000000000

5.4. Таблица итераций для $\ln 7$

$y = (7-1)/(7+1) = 0.75$

k	S (до добавления)	$ t_k $
0	0.000000000000	$1.50 \cdot 10^0$
1	1.500000000000	$2.81 \cdot 10^{-1}$
2	1.781250000000	$9.49 \cdot 10^{-2}$
3	1.876171875000	$3.81 \cdot 10^{-2}$
4	1.914310128348	$1.67 \cdot 10^{-2}$
5	1.930995614188	$7.68 \cdot 10^{-3}$
6	1.938674729830	$3.65 \cdot 10^{-3}$
7	1.942329693525	$1.78 \cdot 10^{-3}$
8	1.944111488327	$8.84 \cdot 10^{-4}$
9	1.944995835011	$4.45 \cdot 10^{-4}$
10	1.945440917389	$2.27 \cdot 10^{-4}$
11	1.945667432527	$1.16 \cdot 10^{-4}$
12	1.945783767748	$6.02 \cdot 10^{-5}$
13	1.945843971224	$3.14 \cdot 10^{-5}$
14	1.945875327202	$1.64 \cdot 10^{-5}$
15	1.945891748543	$8.64 \cdot 10^{-6}$

6. Анализ результатов и выводы

1. Все три алгоритма достигли заданной точности $\varepsilon = 10^{-5}$.
 2. Метод Ньютона для $\sqrt[3]{13}$ показал квадратичную сходимость. Число верных знаков примерно удваивается: после 3-й итерации — ~2 знака, после 4-й — 3-4 знака, после 5-й — машинная точность.
 3. Для $\sqrt[3]{-27}$ начальное приближение $x_0 = 13.5$ далеко от корня ($= 3$). Первые итерации почти линейные; квадратичный режим — с 6-7-й итерации. Всего 8 шагов. Знак восстановлен.
 4. Ряд для $\ln 7$ сходится линейно. При $|y| = 0.75$ потребовалось 15 членов. Погрешность порядка ε .
 5. Фактическая погрешность методов Ньютона на много порядков меньше ε (10^{-15} - 10^{-16}) из-за квадратичной сходимости.
 6. Выбор x_0 существенно влияет на число итераций (особенно для кубического корня).
- Вывод: метод Ньютона эффективен для корней; ряд для логарифма требует больше шагов при $|y|$, близком к 1. Необходимо корректно выбирать x_0 и предусматривать $N_{\max}$.

7. Краткие ответы на контрольные вопросы

1. Итерационный алгоритм — предел последовательности приближений. Результативность — критерий остановки + $N_{\max}$.
2. $F(x)=x^2-a$, $F'=2x \rightarrow x - (x^2-a)/(2x) = (x + a/x)/2$.
3. По разности: $|x_{n+1} - x_n| < \varepsilon$. По невязке: $|F(x_n)| < \varepsilon$.
4. Порядок p : $e_{n+1} \approx C \cdot e_n^p$. У Ньютона $p=2$.
5. x_0 влияет на число итераций. В вар. 6 для $\sqrt[3]{-27}$ $x_0=13.5 \rightarrow 8$ итераций.
6. Ряд $\ln(1+x)$ при больших x медленный. Ряд $y=(c-1)/(c+1)$ всегда $|y| < 1$.
7. Нормализация $s=m \cdot 2^k$ уменьшает $|y|$ и ускоряет сходимость.
8. Погрешность Ньютона обычно $\ll \varepsilon$. Для ряда — порядка ε .
9. $N_{\max}$ защищает от заикливания.
10. Пример: расчёт IRR методом Ньютона.

— Конец отчёта —